

Projeto Prático de POO: P-cub3D (Motor Gráfico 3D Raycasting)

Este projeto adapta o desenvolvimento do motor gráfico 3D baseado no clássico *Wolfenstein 3D* e no algoritmo de *Raycasting* (da especificação do *cub3D*) para o paradigma de **Programação Orientada a Objetos (POO)**. O objetivo é construir uma aplicação gráfica interativa em primeira pessoa que lê um arquivo de descrição de cena/mapa, valida sua estrutura e renderiza a navegação tridimensional em tempo real.

1. Objetivos Pedagógicos de POO

- **Parsing e Tratamento Rigoroso de Erros:** Processar e validar arquivos de cena *.cub*, tratando exceções de parsing e dados inconsistentes de forma segura, garantindo que o programa reporte falhas claramente sem entrar em estado indefinido.
- **Encapsulamento do Estado do Jogo e do Jogador:** Isolar os dados da câmera, vetores de posição, direção e plano de visão do jogador dentro de uma classe bem definida, restringindo alterações de estado a métodos públicos seguros.
- **Padrão MVC e Abstração de Matemática/Gráficos:** Separar os cálculos de projeção matemática (modelo/math) da renderização em tela e controle de entrada do usuário (visão e controladores).
- **Gerenciamento Seguro de Recursos:** Garantir o carregamento, manipulação e desalocação determinística de memórias gráficas (*textures/images*) e descritores de arquivos.

2. Requisitos Funcionais do Sistema

Validação da Cena e do Mapa (*.cub*)

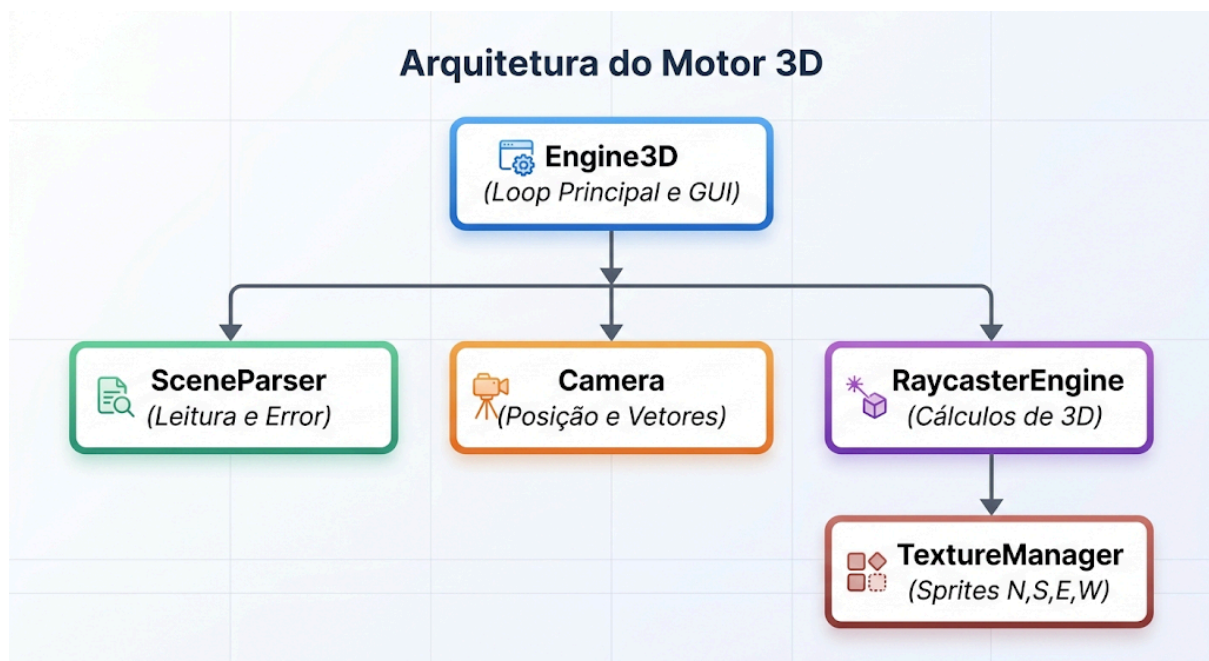
- **Texturas e Cores:** O arquivo deve especificar os caminhos das texturas para as quatro paredes (Norte *NO*, Sul *SO*, Oeste *WE*, Leste *EA*) e os componentes de cor RGB para o Chão (*F*) e Teto (*C*).
- **Validação Estrutural:** O mapa deve ser composto pelos caracteres *0* (espaço vazio), *1* (parede) e exatamente um identificador de posição/orientação inicial do jogador (*N*, *S*, *E* ou *W*).
- **Bordas Fechadas:** O mapa deve ser completamente cercado por paredes (*1*). Espaços em branco devem ser tratados de forma segura para assegurar a integridade do labirinto.

Motor de Renderização 3D (Raycasting)

- **Visão Dinâmica 3D:** Renderização do interior do labirinto a partir da perspectiva em primeira pessoa, aplicando o algoritmo de *Raycasting* para projetar as paredes verticalmente proporcional à distância.

- **Mapeamento de Texturas Orientadas:** Aplicar a textura correspondente à orientação da parede atingida pelo raio (Norte, Sul, Leste ou Oeste).
- **Controle e Movimentação:**
 - Teclas **W**, **A**, **S**, **D** para movimentação da câmera pelo mapa.
 - Setas Esquerda/Direita para rotação suave do ângulo de visão.
 - Encerramento limpo da aplicação ao pressionar **ESC** ou fechar a janela gráfica.

3. Modelo de Arquitetura Orientada a Objetos



Principais Classes de Domínio:

- **Camera** (ou **Player**): Encapsula a posição tridimensional/vetorial (x, y) , os vetores de direção $(\text{dir}_x, \text{dir}_y)$ e o plano da câmera $(\text{plane}_x, \text{plane}_y)$.
- **RaycasterEngine**: Contém o algoritmo matemático puramente focado em projetar os raios a partir da câmera, calcular a distância até a parede e determinar a coordenada da textura.
- **SceneParser**: Classe utilitária responsável por ler o arquivo **.cub**, validar os atributos (RGB e arquivos de imagem) e verificar as regras de fechamento do mapa.
- **TextureManager**: Carrega e mantém em memória os dados das texturas das paredes, fornecendo a amostragem de cores para a renderização.

4. Divisão de Responsabilidades (Equipe de 2 Alunos)

- **Aluno 1: Parser de Cena, Validação do Mapa e Lógica de Entrada (Model/Controller)**
 - Leitura e extração do arquivo de configuração **.cub** (texturas, cores e validação de mapa fechado).
 - Implementação da classe **Camera** e manipulação de eventos de teclado/mouse para cálculo de rotação e deslocamento.
 - Tratamento centralizado de exceções para garantir interrupção segura em caso de mapas ou texturas inválidas.
- **Aluno 2: Motor Raycasting, Gerenciador de Texturas e Renderização (View/Engine)**
 - Implementação do algoritmo de *Raycasting* para cálculo de projeção de paredes e distância focal.
Carregamento de imagens de textura e renderização pixel a pixel das colunas de parede, chão e teto.
 - Gestão da janela gráfica e sincronização da taxa de quadros (*FPS*).

5. Recursos Bônus (Opcionais)

- **Colisão com Paredes:** Implementar física básica para impedir que o jogador atravessasse as paredes do labirinto.
- **Minimapa 2D:** Exibir uma sobreposição de mapa em 2D no canto da tela mostrando a posição do jogador e o campo de visão.
- **Portas e Animações:** Permitir a inclusão de portas abríveis/fecháveis e objetos/inimigos animados (*sprites*) dentro do cenário.
- **Visão por Mouse:** Permitir que a rotação da câmera seja controlada pelo movimento horizontal do mouse.

6. Entregas Esperadas do Projeto

- **Código-Fonte Estruturado:** Projeto desenvolvido em POO, organizando as camadas de visão, matemática, gerenciamento de arquivos e controle.
- **Massa de Testes (.cub):** Conjunto de cenários válidos e cenários corrompidos (mapas abertos, caminhos de textura inexistentes, RGB fora da faixa $\$0\text{--}255\$$) para validação da robustez.
- **Documentação Técnica (README.md):** Explicação da arquitetura OO adotada, diagrama UML resumido, instruções de compilação/execução e relatório de utilização de ferramentas de suporte.
- **Demonstração Prática:** Apresentação da navegação 3D em tempo real, validação da troca de texturas conforme a orientação cardinal e fechamento sem vazamentos de memória.